Clarity built-ins: native vs WebAssembly

Conversions and Hashing & Crypto functions extracted from stacks-core (branch `marf/g3-db-snapshot`), run natively and as a pure (import-free) wasm module. Apple M4 Pro · Rust 1.98 · wasmtime 49.0.2 · wasmi 2.0.0 · time per call, best of 5 × 100 ms samples · (×) = slowdown vs native.

green wasmi + wide-arithmetic is ≥ 10% faster than plain wasmi (6 cells)

Function	Input	native	wasmtime (Cranelift)	wasmi	wasmi + wide-arithmetic	wasmi fuel / call	wasm instructions	code bytes
Conversions								
`buff-to-int-le`	16-byte buffer	2.9 ns	4.8 ns 1.7×	49.0 ns 17.1×	49.9 ns 18.2×	502	554	1,062
`buff-to-int-be`	16-byte buffer	2.3 ns	5.5 ns 2.4×	43.8 ns 18.9×	44.7 ns 18.8×	303	305	599
`buff-to-uint-le`	16-byte buffer	2.8 ns	4.4 ns 1.6×	49.5 ns 17.6×	49.8 ns 18.2×	502	554	1,062
`buff-to-uint-be`	16-byte buffer	2.3 ns	5.2 ns 2.2×	44.4 ns 19.0×	44.9 ns 18.9×	303	305	599
`string-to-int?`	i128 min (40 chars)	70.2 ns	314.8 ns 4.5×	2.33 µs 33.1×	2.28 µs 32.4×	17,307	1,191	2,367
`string-to-uint?`	u128 max (39 chars)	54.7 ns	189.7 ns 3.5×	1.69 µs 30.9×	**909.9 ns** 16.4×	10,678	805	1,601
`int-to-ascii`	i128 min	37.8 ns	63.0 ns 1.7×	515.9 ns 13.6×	**340.8 ns** 9.1×	4,886	1,756	4,119
`int-to-utf8`	i128 min	545.5 ns	598.0 ns 1.1×	4.25 µs 7.8×	3.99 µs 7.4×	112,619	2,802	6,069
`to-ascii?`	uint (u128 max)	64.7 ns	94.3 ns 1.5×	742.8 ns 11.5×	**558.0 ns** 8.4×	7,015	11,940	24,161
`to-ascii?`	32-byte buffer	93.2 ns	105.4 ns 1.1×	1.05 µs 11.3×	1.04 µs 11.2×	11,608	11,928	24,116
`to-ascii?`	standard principal	258.1 ns	433.0 ns 1.7×	3.49 µs 13.5×	3.57 µs 12.9×	30,812	11,958	24,206
`to-ascii?`	contract principal	318.8 ns	494.2 ns 1.6×	3.96 µs 12.4×	4.24 µs 12.4×	35,834	11,907	24,128
`to-ascii?`	utf8 string (56 chars)	127.9 ns	132.2 ns 1.0×	1.26 µs 9.8×	1.35 µs 10.0×	13,627	12,533	25,388
`to-consensus-buff?`	tuple, 5 fields (194 B)	210.8 ns	157.7 ns 0.7×	1.28 µs 6.1×	1.32 µs 6.0×	20,426	1,801	3,874
`from-consensus-buff?`	tuple, 5 fields (194 B)	1.19 µs	1.71 µs 1.4×	29.28 µs 24.6×	31.55 µs 25.2×	225,306	364,693	831,605
`from-consensus-buff?`	tuple, 5 fields (194 B), no regex	1.12 µs	1.60 µs 1.4×	25.14 µs 22.5×	26.78 µs 23.9×	211,264	40,792	95,616
Hashing & Crypto								
`hash160`	32 B	174.6 ns	298.8 ns 1.7×	1.65 µs 9.4×	1.68 µs 8.9×	10,420	8,455	14,409
`sha256`	32 B	29.8 ns	144.9 ns 4.9×	970.3 ns 32.6×	983.6 ns 32.7×	7,380	5,437	9,127
`sha512`	32 B	75.0 ns	178.1 ns 2.4×	1.33 µs 17.7×	1.33 µs 16.1×	9,453	7,529	13,081
`sha512/256`	32 B	73.7 ns	181.1 ns 2.5×	1.28 µs 17.3×	1.31 µs 17.2×	9,261	7,337	12,693
`keccak256`	32 B	161.0 ns	245.3 ns 1.5×	1.78 µs 11.0×	1.82 µs 11.3×	12,594	1,207	2,545
`hash160`	1 KiB	531.1 ns	2.17 µs 4.1×	14.86 µs 28.0×	14.97 µs 28.2×	87,819	8,455	14,409
`sha256`	1 KiB	356.1 ns	2.00 µs 5.6×	14.12 µs 39.7×	14.41 µs 39.4×	84,779	5,437	9,127
`sha512`	1 KiB	579.9 ns	1.38 µs 2.4×	10.07 µs 17.4×	10.25 µs 17.2×	62,260	7,529	13,081
`sha512/256`	1 KiB	579.9 ns	1.39 µs 2.4×	10.10 µs 17.4×	10.20 µs 17.2×	62,068	7,337	12,693
`keccak256`	1 KiB	1.16 µs	1.67 µs 1.4×	13.62 µs 11.8×	13.75 µs 11.8×	85,150	1,207	2,545
`hash160`	64 KiB	22.30 µs	119.33 µs 5.4×	841.47 µs 37.7×	846.20 µs 37.9×	4,960,491	8,455	14,409
`sha256`	64 KiB	22.15 µs	118.03 µs 5.3×	838.28 µs 37.8×	846.18 µs 38.2×	4,957,451	5,437	9,127
`sha512`	64 KiB	32.89 µs	76.40 µs 2.3×	563.00 µs 17.1×	564.53 µs 17.2×	3,385,636	7,529	13,081

Function	Input	native	wasmtime (Cranelift)	wasmi	wasmi + wide-arithmetic	wasmi fuel / call	wasm instructions	code bytes
`sha512/256`	64 KiB	32.87 µs	76.61 µs 2.3×	559.53 µs 17.0×	566.13 µs 17.2×	3,385,444	7,337	12,693
`keccak256`	64 KiB	67.11 µs	97.77 µs 1.5×	812.85 µs 12.1×	814.46 µs 12.1×	4,998,159	1,207	2,545
`secp256k1-recover?`	chain backend	18.55 µs	96.42 µs 5.2×	775.04 µs 41.8×	769.59 µs 40.9×	3,895,071	33,949	83,248
`secp256k1-recover?`	libsecp256k1 everywhere	76.43 µs	96.14 µs 1.3×	759.66 µs 9.9×	758.87 µs 9.8×	3,895,071	33,949	83,248
`secp256k1-verify`	chain backend	18.06 µs	91.14 µs 5.0×	734.24 µs 40.7×	748.78 µs 41.1×	3,734,191	34,907	84,990
`secp256k1-verify`	libsecp256k1 everywhere	72.19 µs	91.03 µs 1.3×	742.39 µs 10.3×	723.96 µs 9.9×	3,734,191	34,907	84,990
`secp256k1-decompress?`	chain backend	2.10 µs	4.82 µs 2.3×	30.50 µs 14.5×	30.70 µs 14.4×	167,080	7,512	18,090
`secp256k1-decompress?`	libsecp256k1 everywhere	4.23 µs	4.81 µs 1.1×	30.49 µs 7.2×	30.97 µs 7.3×	167,080	7,512	18,090
`secp256r1-verify`	Clarity 5+ (prehashed)	133.05 µs	489.49 µs 3.7×	5.33 ms 40.0×	**2.52 ms** 18.8×	21,706,590	27,443	59,511
`secp256r1-verify`	Clarity ≤ 4 (double SHA-256)	133.30 µs	487.49 µs 3.7×	5.30 ms 39.8×	**2.52 ms** 18.6×	21,738,018	32,267	67,367
`ed25519-verify`	64-byte message	23.83 µs	57.57 µs 2.4×	487.56 µs 20.5×	**345.23 µs** 14.4×	2,327,655	21,438	43,405
Hand-written WAT (from the earlier modules)								
`keccak256`	32 B	—	611.5 ns 3.8×	6.88 µs 42.7×	6.99 µs 43.3×	32,174	353	717
`keccak256`	1 KiB	—	4.97 µs 4.3×	57.17 µs 49.5×	54.59 µs 46.7×	250,682	353	717
`keccak256`	64 KiB	—	284.29 µs 4.2×	3.16 ms 47.0×	3.23 ms 48.0×	15,004,226	353	717
`secp256k1-verify`	hand-written WAT	—	553.63 µs 30.7×	4.83 ms 267.5×	4.89 ms 268.2×	30,708,598	1,050	2,019

- **wasmi fuel / call**: fuel consumed by one call under wasmi's default metering (1 per Wasm instruction; `block`/`loop`/`end`/`nop`/`drop`/`return` free; bulk memory ops per byte), default guest. Difference between a 1- and a 3-iteration run, so lazy translation and setup are excluded; includes the benchmark loop's few-instruction checksum fold. Deterministic.
- **Fuel is not proportional to time**: wasmi burns from 4.1 fuel/ns (secp256r1-verify Clarity 5+ (prehashed)) to 26.5 fuel/ns (int-to-utf8 i128 min). Default fuel charges every Wasm instruction 1, including `local.get`/`local.set`/`i32.const` that wasmi's register IR mostly folds away, while multiply-heavy crypto spends real time per instruction — a gas schedule on default fuel would over-charge allocation/copy-heavy functions relative to crypto.
- **wasm instructions / code bytes**: static size of the code each built-in needs: a guest is built with only that case (constant case id, so LTO drops the others) and the empty guest (5,901 instructions, 13,381 bytes: allocator, panic handling) is subtracted. Counted with wasmparser over all function bodies; bytes are body sizes. Hash code size does not depend on input size. to-consensus-buff? is measured against a guest that only runs its benchmark setup (deserializing the input), so the deserializer is not charged to it. Hand-written WAT rows count the whole module except the `bench` loop (keccak256.wat also contains sha3_256, which shares the sponge).
- **Why from-consensus-buff? is 831 KB**: the `regex` crate, used for Clarity name validation — see "The regex issue" below; the "no regex" row is the same function with an equivalent hand-written check. Serializing (to-consensus-buff?) never validates names, hence its small size.
- **wide-arithmetic**: guest built with `-C target-feature=+wide-arithmetic` (64×64→128 multiply) and `curve25519_dalek_bits="64"`; it helps code doing 64-bit-limb or u128 math (secp256r1, ed25519, u128 parsing/printing). Pure-Rust libsecp256k1 uses 32-bit limbs, so secp256k1 is unaffected. Differences under ~5% are run-to-run noise.
- **secp256k1 "chain backend"**: what stacks-core uses on each target — the C library natively, pure-Rust libsecp256k1 in wasm. "libsecp256k1 everywhere" runs the Rust library on all targets; in wasm both rows are the same code (identical fuel and size).
- **Other**: native SHA-2 uses the ARMv8 SHA instructions (as in stacks-common), which inflates the sha256/hash160 ratios. wasmtime's Pulley interpreter was 2–3× slower than wasmi on every case and is no longer measured. Times are measured without fuel metering. The benchmark loop runs inside the module; every engine's result checksum is checked against native before timing. Hand-written WAT rows are compared with the native time of the same function.

The regex issue: from-consensus-buff?

Deserialization validates every tuple key as a ClarityName and every contract principal's name as a ContractName. clarity-types (`representations.rs`) does it with the `regex` crate: `^[a-zA-Z]([a-zA-Z0-9]|[-_!?+<>=/*])*$|^[-+=/*]$|^[<>]=?$` for names, and `^([a-zA-Z](([a-zA-Z0-9]|[-_])){0,127})$|^__transient$` for contract names. These two patterns only need a byte loop, but in a wasm build the `regex` crate costs:

- **Code size**: the regex engine (regex_automata, regex_syntax, aho_corasick, regex, memchr) is ~560 KB of the ~830 KB from-consensus-buff? needs, plus generic code instantiated for it.
- **Per-instance start-up**: the patterns are compiled lazily on first use, inside the guest. Every fresh instance — e.g. one per transaction — pays that on its first name check: ~0.1 ms on Cranelift, ~1 ms on wasmi. The first call on a fresh instance takes 27× (Cranelift) / 15× (wasmi) as long as the second one; without regex the two are within ~20%. (Both calls include the benchmark's one-time setup, a deserialize and type_of, for both variants; the module is compiled before the instance is created.)
- **Per call**: matching itself is cheap: the hand-written check is ~5–15% faster per call.

	regex (as in clarity-types)	hand-written check
wasm instructions	364,693	**40,792**
code bytes	831,605	**95,616**
wasmi fuel / call	225,306	**211,264**
native, per call	1.19 µs	**1.12 µs**
wasmtime (Cranelift), per call	1.71 µs	**1.60 µs**
wasmi, per call	29.28 µs	**25.14 µs**
wasmtime (Cranelift), first call in a fresh instance	94.62 µs	**3.62 µs**
wasmi, first call in a fresh instance	980.33 µs	**54.67 µs**

The hand-written check (`ManualNames` in `kernels/src/consensus.rs`, ~30 lines) accepts exactly the same strings: `kernels/tests/names_equiv.rs` compares it with clarity-types' `ClarityName`/`ContractName` on ~1M strings (exhaustive up to length 4, every character in every position, length boundaries, random). The same replacement would apply to the node: `guarded_string!` in clarity-types.